

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №3

По дисциплине «Логика и основы алгоритмизации в инженерных
задачах»

На тему «Динамические списки»

Выполнил студент группы
24ВВВ2:

Тремасов К. И.

Приняли:

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Научиться реализовывать алгоритмы включающие в себя динамические списки.

Лабораторное задание

1. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).
2. На основе приведенного в задании кода реализуйте структуру данных Очередь.
3. На основе приведенного в задании кода реализовать структуру данных Стек.

Описание метода решения задачи

Для обращения к динамическим данным применим указатели. Набор операций над списком будет включать добавление и удаление элементов, поиск элементов списка.

Используем динамическую реализацию списка. Для создания элемента списка используется структура, в которой объединяются полезная информация и ссылка на следующий элемент списка.

В первом задании переменная `priority` будет хранить приоритет элемента, и в зависимости от приоритета элемент будет помещен на место в списке.

Для реализации очереди используем принцип `first in first out`, для реализации стека используем принцип `last in first out`

Листинг программы

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>
```

```

struct node {
    char inf[256];
    int priority;
    struct node* next;
};

struct node* priority_head = NULL;
struct node* queue_front = NULL;
struct node* queue_rear = NULL;
struct node* stack_top = NULL;

struct node* get_priority_struct(void);
void insert_by_priority(void);
void display_priority(void);
void delete_highest_priority(void);
void destroy_priority_queue(void);

struct node* get_queue_struct(void);
void enqueue(void);
void dequeue(void);
void display_queue(void);
void destroy_queue(void);

struct node* get_stack_struct(void);
void push(void);
void pop(void);
void display_stack(void);
void destroy_stack(void);

void display_menu(void);

int main() {
    setlocale(LC_ALL, "");
    int choice, structure_choice;

    while (1) {
        display_menu();
        scanf_s("%d", &structure_choice);
    }
}

```

```

switch (structure_choice) {
case 1: // Приоритетная очередь
    while (1) {
        printf("\n=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===\n");
        printf("1. Добавить элемент с приоритетом\n");
        printf("2. Просмотреть очередь\n");
        printf("3. Удалить элемент с наивысшим приоритетом\n");
        printf("4. Очистить очередь\n");
        printf("5. Вернуться в главное меню\n");
        printf("Выберите действие: ");
        scanf_s("%d", &choice);

        if (choice == 5) break;

        switch (choice) {
            case 1: insert_by_priority(); break;
            case 2: display_priority(); break;
            case 3: delete_highest_priority(); break;
            case 4: destroy_priority_queue(); break;
            default: printf("Неверный выбор\n");
        }
    }
    break;

case 2: // Обычная очередь
    while (1) {
        printf("\n=== ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ===\n");
        printf("1. Добавить элемент в конец (enqueue)\n");
        printf("2. Удалить элемент из начала (dequeue)\n");
        printf("3. Просмотреть очередь\n");
        printf("4. Очистить очередь\n");
        printf("5. Вернуться в главное меню\n");
        printf("Выберите действие: ");
        scanf_s("%d", &choice);

        if (choice == 5) break;
    }
}

```

```

        switch (choice) {
        case 1: enqueue(); break;
        case 2: dequeue(); break;
        case 3: display_queue(); break;
        case 4: destroy_queue(); break;
        default: printf("Неверный выбор\n");
        }
    }
    break;

```

case 3: // Стек

```

    while (1) {
        printf("\n=== СТЕК (LIFO) ===\n");
        printf("1. Добавить элемент в вершину (push)\n");
        printf("2. Удалить элемент из вершины (pop)\n");
        printf("3. Просмотреть стек\n");
        printf("4. Очистить стек\n");
        printf("5. Вернуться в главное меню\n");
        printf("Выберите действие: ");
        scanf_s("%d", &choice);

        if (choice == 5) break;

        switch (choice) {
        case 1: push(); break;
        case 2: pop(); break;
        case 3: display_stack(); break;
        case 4: destroy_stack(); break;
        default: printf("Неверный выбор\n");
        }
    }
    break;

```

case 4:

```

    // Очистка всех структур перед выходом
    destroy_priority_queue();
    destroy_queue();
    destroy_stack();

```

```

        printf("Выход из программы.\n");
        exit(0);

    default:
        printf("Неверный выбор\n");
    }
}
return 0;
}

void display_menu(void) {
    printf("\n=== ГЛАВНОЕ МЕНЮ ===\n");
    printf("1. Работа с приоритетной очередью\n");
    printf("2. Работа с обычной очередью (FIFO)\n");
    printf("3. Работа со стеком (LIFO)\n");
    printf("4. Выход\n");
    printf("Выберите структуру данных: ");
}

// ===== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ =====

struct node* get_priority_struct(void) {
    struct node* p = (struct node*)malloc(sizeof(struct node));
    char s[256];
    int prio;

    if (p == NULL) {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf_s("%s", s, (unsigned)sizeof(s));
    if (*s == 0) {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }
}

```

```

    printf("Введите приоритет (целое число, чем больше - тем выше приоритет):
");
    scanf_s("%d", &prio);

    strcpy_s(p->inf, sizeof(p->inf), s);
    p->priority = prio;
    p->next = NULL;

    return p;
}

```

```

void insert_by_priority(void) {
    struct node* p = get_priority_struct();
    if (p == NULL) return;

    struct node* current = priority_head;
    struct node* prev = NULL;

    if (priority_head == NULL || p->priority > priority_head->priority) {
        p->next = priority_head;
        priority_head = p;
        printf("Элемент '%s' с приоритетом %d добавлен\n", p->inf, p-
>priority);
        return;
    }
}

```

```

    while (current != NULL && current->priority >= p->priority) {
        prev = current;
        current = current->next;
    }

    prev->next = p;
    p->next = current;
    printf("Элемент '%s' с приоритетом %d добавлен\n", p->inf, p->priority);
}

```

```

void display_priority(void) {

```

```

    struct node* current = priority_head;

    if (priority_head == NULL) {
        printf("Приоритетная очередь пуста\n");
        return;
    }

    printf("\n=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===\n");
    printf("Приоритет | Данные\n");
    printf("-----|-----\n");

    while (current != NULL) {
        printf("%9d | %s\n", current->priority, current->inf);
        current = current->next;
    }
}

void delete_highest_priority(void) {
    if (priority_head == NULL) {
        printf("Приоритетная очередь пуста\n");
        return;
    }

    struct node* temp = priority_head;
    printf("Удален элемент: '%s' с приоритетом %d\n", temp->inf, temp->priority);
    priority_head = priority_head->next;
    free(temp);
}

void destroy_priority_queue(void) {
    struct node* current = priority_head;
    struct node* next;
    int count = 0;

    while (current != NULL) {
        next = current->next;
        free(current);
    }
}

```



```

        current = next;
        count++;
    }

    priority_head = NULL;
    printf("Приоритетная очередь очищена. Удалено элементов: %d\n", count);
}

// ===== ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) =====

struct node* get_queue_struct(void) {
    struct node* p = (struct node*)malloc(sizeof(struct node));
    char s[256];

    if (p == NULL) {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf_s("%s", s, (unsigned)sizeof(s));
    if (*s == 0) {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }

    strcpy_s(p->inf, sizeof(p->inf), s);
    p->next = NULL;

    return p;
}

void enqueue(void) {
    struct node* p = get_queue_struct();
    if (p == NULL) return;

    if (queue_rear == NULL) {

```

```

        queue_front = queue_rear = p;
    }
    else {
        queue_rear->next = p;
        queue_rear = p;
    }
    printf("Элемент '%s' добавлен в конец очереди\n", p->inf);
}

void dequeue(void) {
    if (queue_front == NULL) {
        printf("Очередь пуста\n");
        return;
    }

    struct node* temp = queue_front;
    printf("Удален элемент из начала очереди: '%s'\n", temp->inf);

    queue_front = queue_front->next;

    if (queue_front == NULL) {
        queue_rear = NULL;
    }

    free(temp);
}

void display_queue(void) {
    struct node* current = queue_front;

    if (queue_front == NULL) {
        printf("Очередь пуста\n");
        return;
    }

    printf("\n=== ОЧЕРЕДЬ (FIFO) ===\n");
    printf("Данные\n");
    printf("-----\n");

```

```

    while (current != NULL) {
        printf("%s\n", current->inf);
        current = current->next;
    }
}

void destroy_queue(void) {
    struct node* current = queue_front;
    struct node* next;
    int count = 0;

    while (current != NULL) {
        next = current->next;
        free(current);
        current = next;
        count++;
    }

    queue_front = queue_rear = NULL;
    printf("Очередь очищена. Удалено элементов: %d\n", count);
}

// ===== СТЕК (LIFO) =====

struct node* get_stack_struct(void) {
    struct node* p = (struct node*)malloc(sizeof(struct node));
    char s[256];

    if (p == NULL) {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf_s("%s", s, (unsigned)sizeof(s));
    if (*s == 0) {
        printf("Запись не была произведена\n");
    }
}

```

```

        free(p);
        return NULL;
    }

    strcpy_s(p->inf, sizeof(p->inf), s);
    p->next = NULL;

    return p;
}

void push(void) {
    struct node* p = get_stack_struct();
    if (p == NULL) return;

    p->next = stack_top;
    stack_top = p;
    printf("Элемент '%s' добавлен в вершину стека\n", p->inf);
}

void pop(void) {
    if (stack_top == NULL) {
        printf("Стек пуст\n");
        return;
    }

    struct node* temp = stack_top;
    printf("Удален элемент из вершины стека: '%s'\n", temp->inf);

    stack_top = stack_top->next;
    free(temp);
}

void display_stack(void) {
    struct node* current = stack_top;

    if (stack_top == NULL) {
        printf("Стек пуст\n");
        return;
    }

```

```

    }

    printf("\n=== CTEK (LIFO) ===\n");
    printf("Данные\n");
    printf("-----\n");

    while (current != NULL) {
        printf("%s\n", current->inf);
        current = current->next;
    }
}

void destroy_stack(void) {
    struct node* current = stack_top;
    struct node* next;
    int count = 0;

    while (current != NULL) {
        next = current->next;
        free(current);
        current = next;
        count++;
    }

    stack_top = NULL;
    printf("Стек очищен. Удалено элементов: %d\n", count);
}

```

Результат работы программы

```
Консоль отладки Microsoft Visual Studio

Выберите действие: 2

=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===
Приоритет | Данные
-----
          5 | jkl
          3 | asdf
          1 | nm

=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===
1. Добавить элемент с приоритетом
2. Просмотреть очередь
3. Удалить элемент с наивысшим приоритетом
4. Очистить очередь
5. Вернуться в главное меню
Выберите действие: 5

=== ГЛАВНОЕ МЕНЮ ===
1. Работа с приоритетной очередью
2. Работа с обычной очередью (FIFO)
3. Работа со стеком (LIFO)
4. Выход
Выберите структуру данных: 4
Приоритетная очередь очищена. Удалено элементов: 3
Очередь очищена. Удалено элементов: 0
Стек очищен. Удалено элементов: 0
Выход из программы.

C:\Users\andor\source\repos\3laba\x64\Debug\3laba.exe (процесс 15936) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

```
Консоль отладки Microsoft Visual Studio

=== ОЧЕРЕДЬ (FIFO) ===
Данные
-----
asdf
zxscv
qwew

=== ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ===
1. Добавить элемент в конец (enqueue)
2. Удалить элемент из начала (dequeue)
3. Просмотреть очередь
4. Очистить очередь
5. Вернуться в главное меню
Выберите действие: 5

=== ГЛАВНОЕ МЕНЮ ===
1. Работа с приоритетной очередью
2. Работа с обычной очередью (FIFO)
3. Работа со стеком (LIFO)
4. Выход
Выберите структуру данных: 4
Приоритетная очередь очищена. Удалено элементов: 0
Очередь очищена. Удалено элементов: 3
Стек очищен. Удалено элементов: 0
Выход из программы.

C:\Users\andor\source\repos\3laba\x64\Debug\3laba.exe (процесс 2072) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

```
Консоль отладки Microsoft Visual Studio

=== СТЕК (LIFO) ===
Данные
-----
с
р
а

=== СТЕК (LIFO) ===
1. Добавить элемент в вершину (push)
2. Удалить элемент из вершины (pop)
3. Просмотреть стек
4. Очистить стек
5. Вернуться в главное меню
Выберите действие: 5

=== ГЛАВНОЕ МЕНЮ ===
1. Работа с приоритетной очередью
2. Работа с обычной очередью (FIFO)
3. Работа со стеком (LIFO)
4. Выход
Выберите структуру данных: 4
Приоритетная очередь очищена. Удалено элементов: 0
Очередь очищена. Удалено элементов: 0
Стек очищен. Удалено элементов: 3
Выход из программы.

C:\Users\andor\source\repos\3laba\x64\Debug\3laba.exe (процесс 19600) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно: _
```

Вывод: Реализовали алгоритм, включающий в себя структуры приоритетной очереди, очереди и стека. Каждая структура данных работает независимо и демонстрирует соответствующий принцип обработки элементов. Научились реализовывать алгоритмы включающие в себя динамические списки и динамические структуры данных с эффективным распределением памяти.